

PARTIE II — Classes, Objets et Syntaxe Fondamentale

7. Les méthodes

7.1. Définition d'une méthode

1. Idée simple

Une **méthode** est une **fonction placée dans une classe**.

Elle décrit ce qu'un **objet peut faire**.

2. Exemple simple

```
class Personne:
    def dire_bonjour(self):
        print("Bonjour !")
```

Ici, `dire_bonjour()` est une **méthode** de la classe `Personne`.

3. Image mentale 🧠

- **Attribut** = ce que l'objet a
- **Méthode** = ce que l'objet fait

Exemple :

```
class Voiture:
    couleur = "rouge"          # attribut

    def demarrer(self):        # méthode
        print("La voiture démarre")
```

4. À retenir

Une **méthode** est une action appartenant à un objet.

Formule simple :

Objet = données + actions

Attributs = données

Méthodes = actions

7.2. Méthode d'instance

1. Idée simple

Une **méthode d'instance** est une méthode qui agit sur **un objet précis**.

Elle utilise les données de cet objet.

2. Mot-clé essentiel

En Python, elle reçoit souvent `self` :

```
class Personne:
    def se_presenter(self):
        print("Je suis une personne")
```

`self` représente **l'objet actuel**.

3. Exemple clair

```
class Personne:
    def __init__(self, nom):
        self.nom = nom

    def se_presenter(self):
        print("Je m'appelle", self.nom)
```

```
p1 = Personne("Awa")
```

```
p1.se_presenter()
```

Résultat :

```
Je m'appelle Awa
```

4. Image mentale 🧠

Une méthode d'instance répond à :

“Que peut faire **cet objet précis** avec ses propres données ?”

5. À retenir

- **Méthode d'instance** = action liée à **un objet**
- Elle peut lire ou modifier les **attributs de cet objet**
- En Python, elle utilise généralement `self`

7.3. Méthode de classe

1. Idée simple

Une **méthode de classe** est une méthode liée à la **classe elle-même**, pas à un objet précis.

Elle sert à agir sur des informations communes à toute la classe.

2. Mot-clé essentiel

En Python, on utilise `@classmethod` et `cls` :

```
class Personne:
    nombre = 0

    @classmethod
    def compter(cls):
        print("Nombre de personnes :", cls.nombre)
```

`cls` représente **la classe actuelle**.

3. Exemple clair

```
class Personne:
    nombre = 0

    def __init__(self):
```

```
Personne.nombre += 1
```

```
@classmethod  
def afficher_nombre(cls):  
    print(cls.nombre)
```

```
p1 = Personne()
```

```
p2 = Personne()
```

```
Personne.afficher_nombre()
```

Résultat :

2

4. Image mentale 🧠

Une méthode de classe répond à :

“Que peut faire la **classe entière** avec ses données communes ?”

5. À retenir

- **Méthode d’instance** → liée à **un objet**
- **Méthode de classe** → liée à **la classe**
- En Python, elle utilise `@classmethod` et `cls`

7.4. Méthode statique

1. Idée simple

Une **méthode statique** est une méthode placée dans une classe, mais qui n’utilise ni l’objet ni la classe.

Elle sert souvent pour une **fonction utile**, rangée dans la classe.

2. Mot-clé essentiel

En Python, on utilise `@staticmethod` :

```
class Calcul:

    @staticmethod
    def additionner(a, b):

        return a + b
```

Ici, `additionner()` ne dépend ni de `self`, ni de `cls`.

3. Exemple clair

```
resultat = Calcul.additionner(3, 5)
print(resultat)
```

Résultat :

8

4. Image mentale 🧠

Une méthode statique répond à :

“Quelle fonction utile peut être rangée dans cette classe ?”

5. À retenir

- **Méthode d'instance** → utilise `self`
- **Méthode de classe** → utilise `cls`
- **Méthode statique** → n'utilise ni `self`, ni `cls`

7.5. Méthode publique

1. Idée simple

Une **méthode publique** est une méthode que l'on peut utiliser **depuis l'extérieur de la classe**.

Elle fait partie de l'interface normale de l'objet.

2. Exemple clair

```
class Compte:
```

```
def afficher_solde(self):  
    print("Solde disponible")
```

Ici, `afficher_solde()` est une **méthode publique**.

On peut l'appeler directement :

```
compte = Compte()  
compte.afficher_solde()
```

3. Image mentale 🧠

Une méthode publique, c'est comme un **bouton visible** sur un objet.

Tu peux appuyer dessus depuis l'extérieur.

4. À retenir

- **Méthode publique** = méthode accessible normalement
- Elle est faite pour être utilisée par d'autres parties du programme
- En Python, par défaut, une méthode est **publique** ☒

7.6. Méthode privée

1. Idée simple

Une **méthode privée** est une méthode destinée à être utilisée **à l'intérieur de la classe seulement**.

Elle sert souvent à cacher un détail interne.

2. Exemple clair

```
class Compte:  
    def __verifier_solde(self):  
        print("Vérification du solde")  
  
    def retirer(self):  
        self.__verifier_solde()
```

```
print("Retrait effectué")
```

Ici, `__verifier_solde()` est une **méthode privée**.

Elle est utilisée par `retirer()`, mais elle n'est pas faite pour être appelée directement de l'extérieur.

3. Image mentale 🧠

Une méthode privée, c'est comme un **mécanisme caché** dans une machine.

L'utilisateur voit le bouton, mais pas tout le système interne.

4. À retenir

- **Méthode publique** → utilisée depuis l'extérieur
- **Méthode privée** → utilisée à l'intérieur de la classe
- En Python, on indique souvent une méthode privée avec `__` devant son nom ☒

7.7. Méthode protégée

1. Idée simple

Une **méthode protégée** est une méthode destinée à être utilisée :

- dans la **classe elle-même**
- dans ses **classes enfants**

Elle n'est pas faite pour être utilisée directement depuis l'extérieur.

2. Exemple clair

```
class Animal:
    def _respirer(self):
        print("L'animal respire")
```

```
class Chien(Animal):
    def vivre(self):
```

```
self._respirer()
```

Ici, `_respirer()` est une **méthode protégée**.

Elle peut être utilisée par `Animal` et par `Chien`.

3. Image mentale 🧠

Une méthode protégée, c'est comme un **outil réservé à la famille**.

Pas pour tout le monde, mais disponible pour les classes liées.

4. À retenir

- **Méthode publique** → accessible partout
- **Méthode privée** → utilisée surtout dans la classe
- **Méthode protégée** → utilisée dans la classe et ses enfants
- En Python, on indique souvent une méthode protégée avec `_` devant son nom ☒

7.8. Paramètres d'une méthode

1. Idée simple

Les **paramètres** sont des informations qu'on donne à une méthode pour qu'elle fasse son travail.

Une méthode peut donc recevoir des valeurs.

2. Exemple clair

```
class Personne:
    def saluer(self, nom):
        print("Bonjour", nom)
```

Ici, `nom` est un **paramètre**.

On l'utilise ainsi :

```
p = Personne()
p.saluer("Awa")
```


Résultat :

Bonjour Awa

3. Image mentale

Une méthode, c'est comme une machine.

Les **paramètres** sont les éléments qu'on met dans la machine pour obtenir un résultat.

4. À retenir

- **Méthode** = action
- **Paramètres** = informations données à l'action
- En Python, `self` représente l'objet, les autres paramètres sont les données utiles ☒

7.9. Valeur de retour d'une méthode

1. Idée simple

La **valeur de retour** est le résultat qu'une méthode **renvoie** après son travail.

En Python, on utilise `return`.

2. Exemple clair

```
class Calcul:
    def doubler(self, nombre):
        return nombre * 2
```

Ici, la méthode `doubler()` renvoie un résultat.

```
c = Calcul()
resultat = c.doubler(4)
```

```
print(resultat)
```

Résultat :

3. Image mentale

Une méthode avec `return`, c'est comme une machine :

tu donnes une valeur → elle travaille → elle te rend un résultat.

4. À retenir

- `print()` → affiche un résultat
- `return` → renvoie un résultat utilisable
- Une méthode peut donc **produire une valeur** ☒

7.10. Méthodes avec effets de bord

1. Idée simple

Une **méthode avec effet de bord** est une méthode qui **modifie quelque chose**.

Elle ne fait pas seulement un calcul : elle change l'état d'un objet, affiche un message, écrit dans un fichier, etc.

2. Exemple clair

```
class Compte:
```

```
    def __init__(self, solde):
        self.solde = solde
```

```
    def deposter(self, montant):
        self.solde += montant
```

Ici, `deposer()` a un **effet de bord** :

```
compte = Compte(100)
```

```
compte.deposer(50)
```

```
print(compte.solde)
```

Résultat :

150

La méthode a modifié `solde`.

3. Image mentale 🧠

Une méthode avec effet de bord, c'est comme une action réelle : après l'avoir faite, quelque chose a changé.

4. À retenir

- **Sans effet de bord** → renvoie seulement une valeur
- **Avec effet de bord** → modifie un état ou produit une action visible
- Exemple fréquent : modifier un attribut d'objet ☒

7.11. Méthodes pures

1. Idée simple

Une **méthode pure** est une méthode qui :

- renvoie un résultat
- ne modifie rien autour d'elle

Elle ne change pas l'état de l'objet.

2. Exemple clair

```
class Calcul:  
    def doubler(self, nombre):  
        return nombre * 2
```

Ici, `doubler()` est une **méthode pure** :

```
c = Calcul()  
resultat = c.doubler(5)
```

```
print(resultat)
```

Résultat :

10

La méthode calcule, mais ne modifie aucun attribut.

3. Image mentale 🧠

Une méthode pure, c'est comme une calculatrice :

tu donnes une valeur → elle calcule → elle rend un résultat, sans rien changer.

4. À retenir

- **Méthode pure** → produit un résultat
- Elle ne modifie pas l'objet
- Elle est plus facile à tester et à comprendre ☒

7.12. Méthodes de lecture

1. Idée simple

Une **méthode de lecture** sert à **lire une donnée d'un objet** sans la modifier.

Elle permet d'accéder proprement à un attribut.

2. Exemple clair

```
class Compte:
    def __init__(self, solde):
        self.__solde = solde

    def get_solde(self):
        return self.__solde
```

Ici, `get_solde()` est une **méthode de lecture**.

```
compte = Compte(100)
print(compte.get_solde())
```

Résultat :

100

3. Image mentale 🧠

Une méthode de lecture, c'est comme une **fenêtre** :
elle permet de voir une donnée, sans la toucher.

4. À retenir

- **Méthode de lecture** = lire une valeur
- Elle ne modifie pas l'objet
- On l'appelle souvent **getter** en anglais ☒

7.13. Méthodes de modification

1. Idée simple

Une **méthode de modification** sert à **changer une donnée d'un objet**.
Elle modifie souvent la valeur d'un attribut.

2. Exemple clair

```
class Compte:
    def __init__(self, solde):
        self.__solde = solde

    def set_solde(self, nouveau_solde):
        self.__solde = nouveau_solde
```

Ici, `set_solde()` est une **méthode de modification**.

```
compte = Compte(100)
compte.set_solde(200)
```

Le solde est maintenant modifié.

3. Image mentale

Une méthode de modification, c'est comme un **bouton de réglage** : elle permet de changer une valeur interne.

4. À retenir

- **Méthode de lecture** → lit une valeur
- **Méthode de modification** → change une valeur
- On l'appelle souvent **setter** en anglais ☒

7.14. Méthodes utilitaires

1. Idée simple

Une **méthode utilitaire** sert à faire une **petite tâche pratique** pour aider la classe.

Elle n'est pas toujours liée directement à l'état de l'objet.

2. Exemple clair

```
class Texte:
    @staticmethod
    def nettoyer(phrase):
        return phrase.strip().lower()
```

Ici, `nettoyer()` est une **méthode utilitaire** :

```
resultat = Texte.nettoyer("  BONJOUR  ")
print(resultat)
```

Résultat :

bonjour

3. Image mentale

Une méthode utilitaire, c'est comme un **petit outil rangé dans une boîte**.

On l'utilise quand on en a besoin pour simplifier le travail.

4. À retenir

- **Méthode utilitaire** = méthode pratique d'aide
- Elle sert souvent à nettoyer, convertir, vérifier ou calculer
- Elle est souvent écrite comme une **méthode statique** avec `@staticmethod` ☒

7.15. Bonnes pratiques de nommage des méthodes

1. Idée simple

Le nom d'une méthode doit dire clairement **ce que la méthode fait**.

Une méthode représente souvent une **action**, donc son nom commence souvent par un **verbe**.

2. Exemples clairs

☒ Bons noms :

```
def calculer_total(self):  
    pass
```

```
def afficher_solde(self):  
    pass
```

```
def verifier_mot_de_passe(self):  
    pass
```

☒ Noms moins clairs :

```
def total(self):  
    pass
```

```
def action(self):  
    pass
```

```
def test(self):  
    pass
```

3. Image mentale 🧠

Le nom d'une méthode doit être comme une **étiquette sur un bouton** :
on doit comprendre l'action avant même de lire le code.

4. À retenir

- Utiliser un nom **clair**
- Commencer souvent par un **verbe**
- Éviter les noms vagues : faire, action, traiter, test
- En Python, écrire souvent en snake_case : calculer_total()
☒